

Reviewer Pack — Minimal Local Zeta Function Order and Frege Proof Depth Ineq...

Entry 13665578f781 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 18:39:52 UTC

Minimal Local Zeta Function Order and Frege Proof Depth Inequality

Entry ID: 13665578f781 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 18:39:52 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry (Local Zeta Functions) **Field B** (complexity object): Resolution Proofs (Frege Proof Complexity)

Statement:

For every d -regular graph G , the minimal local zeta function order ($\text{lzf}(G)$) of its associated Tseitin formula φ_G is linearly correlated with its Frege proof depth $d(\varphi_G)$, such that $\text{lzf}(G) = \Theta(d(\varphi_G))$.

Rationale (proposer's reasoning):

The minimal local zeta function order measures the complexity of the arithmetic geometry associated with the graph, and it has been used in other mathematical contexts to study the structure of spaces. If this invariant could be shown to correlate with Frege proof depth, it would provide a novel perspective on the complexity of resolution proofs.

Taxonomy category: resolution-proof-complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c8502b8217cd577e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given d -regular graph G , if the computed local zeta function order $\text{lzf}(\varphi_G)$ is within a factor of 2 from $d(\varphi_G)$, or if the correlation coefficient between $\text{lzf}(G)$ and $d(\varphi_G)$ exceeds 0.8 using 30 random seeds, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 12 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Local Zeta Function Order AND Frege Proof Depth - Tseitin Formula $lzf(G)$ INCLUSIVE Frege proof depth $d(\phi_G)$ - Arithmetic Geometry local zeta functions CORRELATION Resolution Proofs proof complexity

Top relevant hits considered: - [http://arxiv.org/abs/math/0311029v1] Zeta functions over zeros of general zeta and L -functions - [http://arxiv.org/abs/0903.1238v3] Motivic Zeta Functions for Curve Singularities - [http://arxiv.org/abs/1712.05169v12] Zeros and the functional equation of the quadrilateral zeta function - [http://arxiv.org/abs/2604.05701v1] Measurement of the CKM angle γ in $B^\pm \rightarrow D(\rightarrow K_S^0 h'^+ h'^-) h^\pm$ dec - [http://arxiv.org/abs/2305.01463v2] Test of lepton flavour universality using $B^0 \rightarrow D^{*-} \tau^+ \nu_\tau$ decays with hadronic τ channels - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/math/0309425v2] Algebraic Aspects of Multiple Zeta Values - [http://arxiv.org/abs/2112.04791v2] Kähler-Einstein metrics and Archimedean zeta functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if n % d != 0:
            raise ValueError("Graph size must be a multiple of the degree")
        graph = {i: [] for i in range(n)}
        edges_added = set()
        while len(edges_added) < (n * d) // 2:
            u = random.randint(0, n - 1)
            v = random.randint(0, n - 1)
            if u != v and (u, v) not in edges_added and (v, u) not in edges_added:
                graph[u].append(v)
                graph[v].append(u)
                edges_added.add((u, v))
        return graph

    def tseitin_formula(graph):
```

```

n = len(graph)
clauses = []
literals = {}
for i in range(n):
    literals[i] = f"x{i}"

def add_clause(clause):
    clauses.append(clause)

for i in range(n):
    clause = [f"~{literals[i]}"]
    for j in graph[i]:
        clause.append(literals[j])
    add_clause(clause)

for i in range(n):
    for j in range(i + 1, n):
        for k in range(j + 1, n):
            clause = [f"~{literals[i]}", f"~{literals[j]}", literals[k]]
            add_clause(clause)
            clause = [f"~{literals[i]}", f"~{literals[k]}", literals[j]]
            add_clause(clause)
            clause = [f"~{literals[j]}", f"~{literals[k]}", literals[i]]
            add_clause(clause)

return clauses

def frege_proof_depth(clauses):
    n = len(clauses)
    if n == 0:
        return 0
    depth = [1] * n
    for i in range(n):
        for clause in clauses[i]:
            if clause.startswith("~"):
                j = int(clause[1:])
                depth[j - 1] += 1
    return max(depth)

def local_zeta_function_order(graph):
    n = len(graph)
    zeta_values = [0] * (n + 1)
    for i in range(n):
        zeta_values[i + 1] = sum(1 / (i + 1) for i in graph[i])
    return max(zeta_values)

def correlation_coefficient(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    std_x = math.sqrt(sum((x[i] - mean_x) ** 2 for i in range(n)) / n)
    std_y = math.sqrt(sum((y[i] - mean_y) ** 2 for i in range(n)) / n)
    return cov_xy / (std_x * std_y)

results = []

```

```

for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    d = random.randint(2, min(n - 1, 6))
    graph = generate_d_regular_graph(n, d)
    clauses = tseitin_formula(graph)
    proof_depth = frege_proof_depth(clauses)
    zeta_order = local_zeta_function_order(graph)
    results.append((zeta_order, proof_depth))

metric_value = correlation_coefficient([x for x, _ in results], [y for _, y in results])
conjecture_holds = abs(metric_value) > 0.8
counterexample = "" if conjecture_holds else "correlation_coefficient"

return {
    "metric_name": "Correlation Coefficient",
    "metric_value": metric_value,
    "instances_tested": len(results),
    "n_max": max(n for _, _ in results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {metric_value}, \"instances_tested\": {len(results)}, \"n_max\": {n_max}, \"conjecture_holds\": {conjecture_holds}, \"counterexample\": {counterexample}}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d91cc44d.py", line 119, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d91cc44d.py", line 95, in run_trial

```

```
graph = generate_d_regular_graph(n, d)
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d91cc44d.py", line 23, in generate_d_re
    raise ValueError("Graph size must be a multiple of the degree")
ValueError: Graph size must be a multiple of the degree
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Investigate and fix the error causing the test to crash, then re-run the test with a valid graph to evaluate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21251
2	preregistration	ollama_remote	glm4:latest	0	0	10058
3	novelty	ollama_remote	glm4:latest	0	0	10028
4	novelty	ollama_remote	glm4:latest	0	0	9853
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19491
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14502
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13941
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15622
9	judge	ollama_remote	glm4:latest	0	0	14035

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 128782 ms total latency. Provider mix: {'ollama remote': 9}

(full prompt+response transcripts available in [research/audit/13665578f781.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/13665578f781.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp sandbox/test *.py` (per-cycle)

Replay tarball: `research/replay/13665578f781.tar.gz` (if generated)